

test-coverage

VisionEngine — Test Coverage Ledger

Round: 297 (deep-doc + Challenge enrichment, mirror round-220 template) **Date:** 2026-05-19 **Scope:** every exported symbol in pkg/{analyzer,graph,llmvision,opencv,config,i18n,remote} mapped to the test (or Challenge) that exercises it with positive runtime evidence per Article XI §11.9.

This ledger is the symbol→test "honest map" mandated by CONST-035 + CONST-050(B). An exported symbol present here without a "test that proves real behaviour" reference is a coverage gap; an entry whose listed test does not actually call the symbol is a bluff and MUST be rewritten.

Verification reproducer

```
cd dependencies/HelixDevelopment/VisionEngine
go test -race -count=1 ./...
./challenges/scripts/
    visionengine_describe_challenge.sh          # normal,
    exit 0
./challenges/scripts/visionengine_describe_challenge.sh --
    mutate # paired-mutation, exit 99
```

Symbol → Test ledger

pkg/analyzer

Symbol	Covered by
Analyzer (interface)	stub_test.go, runner exercises StubAnalyzer impl
VideoProcessor (interface)	declared-only contract — no impl yet (TODO future round)
StubAnalyzer	stub_test.go, runner AnalyzeScreen checks

Symbol	Covered by
NewStubAnalyzer	stub_test.go::TestStubAnalyzer_*, runner
NewStubAnalyzerWithProvider	stub_test.go::TestStubAnalyzer_WithProvider
StubAnalyzer.SetTranslator	i18n_callsites_test.go
StubAnalyzer.AnalyzeScreen	stub_test.go + runner (positive + negative-path empty buffer)
StubAnalyzer.CompareScreens	stub_test.go::TestStubAnalyzer_CompareScreens
StubAnalyzer.DetectElements	stub_test.go::TestStubAnalyzer_DetectElements
StubAnalyzer.DetectText	stub_test.go::TestStubAnalyzer_DetectText
StubAnalyzer.IdentifyScreen	stub_test.go::TestStubAnalyzer_IdentifyScreen
StubAnalyzer.DetectIssues	stub_test.go::TestStubAnalyzer_DetectIssues
ErrEmptyScreenshot	runner check 2 (negative-path proof)
ErrAnalysisFailed, ErrComparisonFailed, ErrDetectionFailed, ErrIdentificationFailed	sentinel — referenced by impl returns; covered transitively
Rect.Contains, .Overlaps, .Area, type-only	stub_test.go + runner check 5
UIElement, TextRegion, ScreenAnalysis, ScreenDiff, ScreenIdentity, Action, VisualIssue, Size, KeyFrame	type-only — composed in runner + graph tests
LLMVisionProvider (interface)	stub_test.go::TestStubAnalyzer_WithProvider
fingerprint (unexported helper)	exercised via AnalyzeScreen — fingerprint shown in runner output

pkg/graph

Symbol	Covered by
NavigationGraph (interface)	graph_test.go, runner check 3 + 4
NewNavigationGraph	graph_test.go::TestNewNavigationGraph, runner
*navigationGraph.AddScreen	graph_test.go, runner
*navigationGraph.AddTransition	graph_test.go, runner + paired-mutation Challenge
*navigationGraph.CurrentScreen/ SetCurrent	graph_test.go, runner
navigationGraph.PathTo	graph_test.go::TestPathTo_, runner check 3
*navigationGraph.Screens/ Transitions	graph_test.go, runner export checks
ExportDOT, ExportJSON, ExportMermaid	export_test.go, runner check 4 (all three back-ends) graph_test.go (negative-path assertions)

Symbol	Covered by
ErrScreenNotFound, ErrNoPath, ErrEmptyGraph, ErrSelfTransition, ErrDuplicateScreen	
Transition	composed by PathTo — runner inspects len(path)==2
stress / automation / integra- tion / security suites	graph_{stress,automation,integration,security} _test.go

pkg/llmvision

Symbol	Covered by
VisionProvider (inter- face)	provider_test.go
OpenAIVisionProvider	openai.go impl + provider_test.go con- struction
AnthropicVisionProvider	anthropic.go impl + provider_test
GeminiVisionProvider	gemini.go impl + provider_test
QwenVLProvider	qwen.go impl + provider_test
KimiVisionProvider	kimi.go impl + provider_test
StepGUIVisionProvider	stepgui.go impl + provider_test
OllamaVisionProvider	ollama_test.go (offline-skip pattern)
AsticaVisionProvider	astica_test.go
FallbackChain	provider_test.go::TestFallback*

pkg/opencv

Symbol	Covered by
stub.go (no-vision tag)	compiled in default-build path of go test ./...
vision.go (vision tag)	gated by -tags vision — not in standard CI run

pkg/config

Symbol	Covered by
LoadFromEnv, Validate	config_test.go (real env-var parsing, er- ror-classification)
i18n-routed validation errors	config_test.go + pkg/llmvision callsites

pkg/i18n

Symbol	Covered by
Translator (interface)	translator_test.go
NoopTranslator	translator_test.go::TestNoopTranslator, runner check 6
bundled active.en.yaml	consumed by analyzer/i18n_defaults.go + llmvision/i18n_defaults.go

pkg/remote

Symbol	Covered by
VisionPool	covered by remote_test.go; round-40 SSH wiring closed sentinel bluffs
EnsureReady, Shutdown	round-40 anti-bluff close-out (commit 8dbf6fd)

Anti-bluff invariants on this ledger

1. Every test referenced above MUST end the relevant assertion path in `require.NoError` or `assert.Equal` on a real return value — never on a sentinel "function was called" probe.
2. Runner-side checks print bilingual (5-locale) labels per CONST-046 so captured artefacts cannot be English-only smoke.
3. Paired-mutation Challenge (`--mutate`) MUST exit 99 (genuine regression detection). A 0 from the mutate path proves the Challenge is a bluff.
4. Coverage gaps (e.g. VideoProcessor interface has no impl yet) are listed honestly above — pretending they're covered would itself be a CONST-035 violation.

Evidence (round 297)

```
$ go test -race -count=1 ./...
ok  digital.vasic.visionengine/pkg/analyzer    1.013s
ok  digital.vasic.visionengine/pkg/config      1.015s
ok  digital.vasic.visionengine/pkg/graph       1.398s
ok  digital.vasic.visionengine/pkg/i18n        1.010s
ok  digital.vasic.visionengine/pkg/llmvision   1.093s
ok  digital.vasic.visionengine/pkg/opencv      1.021s
ok  digital.vasic.visionengine/pkg/remote      1.127s

$ ./challenges/scripts/visionengine_describe_challenge.sh ; echo
```

```
$?  
[visionengine_describe_challenge] PASS – runtime evidence captured  
per $11.9  
0
```

```
$ ./challenges/scripts/visionengine_describe_challenge.sh --  
mutate ; echo $?  
[visionengine_describe_challenge] mutation correctly detected –  
Challenge is genuine (exit 99)  
99
```